

Các Cận Thời Gian Cho Bài Toán Chọn

Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, Robert E. Tarjan

Nguồn gốc: Time Bounds for Selection

English Materials/Time Bounds For Selection - Blum_Floyd_Pratt_Rivest_Tarjan.pdf

Abstract

Số phép so sánh cần để chọn phần tử nhỏ thứ i trong n số được chứng minh là bị chặn trên bởi một hàm tuyến tính theo n , thông qua phân tích một thuật toán chọn mới, PICK. Cụ thể, không bao giờ cần quá $5.4305n$ phép so sánh. Cận này được cải thiện khi i nằm gần hai đầu của dãy thứ hạng, và một cận dưới mới cho số phép so sánh cần thiết cũng được chứng minh.

1 Giới thiệu

Bài báo này trình bày một thuật toán chọn mới, PICK, và từ phân tích hiệu quả của nó suy ra kết quả đáng chú ý rằng chi phí chọn bị chặn trên bởi một hàm tuyến tính theo số phần tử đầu vào. Ngoài ra, bài báo cũng chứng minh một cận dưới mới cho chi phí chọn.

Bài toán chọn có thể được minh họa rõ nhất bằng việc tính trung vị. Tổng quát hơn, ta muốn chọn phần tử nhỏ thứ i trong một tập gồm n số phân biệt, hoặc phần tử có thứ hạng gần nhất với một mức phần trăm cho trước.

Sự quan tâm đến bài toán này có thể truy về các cuộc thi đấu thể thao, đặc biệt là thiết kế giải đấu quần vợt để chọn người giỏi nhất và người giỏi nhì. Năm 1883, Lewis Carroll công bố một bài viết phê phán cách xác định người giỏi nhì trong một giải “loại trực tiếp”: người thua trận chung kết thường không nhất thiết là người giỏi nhì, vì bất kỳ người chơi nào chỉ thua người giỏi nhất cũng có thể là người giỏi nhì. Khoảng năm 1930, Hugo Steinhaus đưa vấn đề này vào phạm vi độ phức tạp thuật toán bằng câu hỏi: cần ít nhất bao nhiêu trận để xác định đúng cả người giỏi nhất lẫn người giỏi nhì trong n đấu thủ? Năm 1932, J. Schreier chứng minh rằng không cần quá

$$n + \lceil \log_2 n \rceil - 2$$

trận, và năm 1964, S. S. Kislitsin chứng minh số này cũng là cần thiết. Phương pháp của Schreier dùng một giải loại trực tiếp để tìm người thắng, rồi một giải loại trực tiếp thứ hai giữa các đấu thủ đã thua người thắng để tìm người về nhì.

Với $i > 2$, số trận tối thiểu cần để chọn người giỏi thứ i trong n đấu thủ chỉ được biết với các giá trị nhỏ của n . Thủ tục chọn tổng quát tốt nhất trước đó là của Hadian và Sobel, cần không quá

$$n - i + (i - 1)\lceil \log_2(n - i + 2) \rceil$$

trận. Họ tạo một giải loại trực tiếp gồm $n - i + 2$ người, rồi lần lượt loại $i - 1$ người “quá giỏi” để có thể là người giỏi thứ i , bằng kỹ thuật chọn có thay thế.

Trong tài liệu không có ký hiệu thống nhất cho “phần tử giỏi thứ i ”. Bài báo dùng hai toán tử sau.

- $i \circ S$, đọc là “phần tử thứ i của S ”, là phần tử nhỏ thứ i của S , với $1 \leq i \leq |S|$. Độ lớn của $i \circ S$ tăng khi i tăng. Khi S đã rõ, ta viết ngắn là $i \circ$.
- $x \rho S$, đọc là “hạng của x trong S ”, là hạng của x trong S , sao cho

$$(x \rho S) \circ S = x.$$

Gọi $f(i, n)$ là chi phí tối thiểu trong trường hợp xấu nhất, tức số phép so sánh nhị phân cần để chọn $i \circ$, với $|S| = n$. Ta cũng dùng ký hiệu

$$F(\alpha) := \limsup_{n \rightarrow \infty} \frac{f(\lfloor \alpha(n-1) \rfloor + 1, n)}{n}, \quad 0 \leq \alpha \leq 1,$$

để đo độ khó tương đối của việc tính các mức phần trăm.

Trong Phần 2, kết quả chính $f(i, n) = O(n)$ được chứng minh bằng cách phân tích thuật toán chọn cơ bản PICK. Trong Phần 3, PICK được tinh chỉnh để thu được các cận chặt nhất của bài báo:

$$\max_{0 \leq \alpha \leq 1} F(\alpha) \leq 5.4303 \quad (1)$$

và

$$F(\alpha) \leq 1 + 4.4303 \frac{\alpha}{\beta} + 10.861 \alpha [\log_2(\beta/\alpha)], \quad 0 < \alpha < \beta, \quad (2)$$

trong đó $\beta = 0.203688 \dots$. Trong Phần 4, bài báo suy ra cận dưới

$$F(\alpha) \geq 1 + \min(\alpha, 1 - \alpha), \quad 0 \leq \alpha \leq 1. \quad (3)$$

Không có bằng chứng nào cho thấy các bất đẳng thức (1)–(3) là tốt nhất có thể; tác giả phỏng đoán rằng chúng còn có thể được cải thiện đáng kể.

2 Thuật toán chọn mới PICK

Trong phần này ta mô tả thuật toán cơ bản và chứng minh $f(i, n) = O(n)$. Giả sử cần chọn $i \circ S$, với $|S| = n$.

PICK hoạt động bằng cách lần lượt loại khỏi S các tập con mà mọi phần tử trong đó đã biết là quá lớn hoặc quá nhỏ để có thể là $i \circ$, cho đến khi chỉ còn lại $i \circ$. Mỗi tập con bị loại chứa ít nhất một phần tử số phần tử còn lại. PICK khá giống thuật toán FIND của Hoare, ngoại trừ việc phần tử m dùng để phân hoạch S được chọn cẩn thận hơn.

Thuật toán được mô tả bằng ba hàm phụ $b(i, n)$, $c(i, n)$, và $d(i, n)$; khi không gây nhầm lẫn, ta bỏ qua danh sách đối số của chúng. Vì trọng tâm là tính chất tiệm cận của PICK, các chi tiết cho trường hợp $n \bmod c \neq 0$ sẽ được bỏ qua ở đây và được xử lý trong Phần 3.

Thuật toán PICK. Chọn $i \circ S$, với $|S| = n$ và $1 \leq i \leq n$.

1. Chọn một phần tử $m \in S$.

(a) Sắp S thành n/c cột, mỗi cột có độ dài c , rồi sắp xếp từng cột.

(b) Gọi T là tập gồm n/c phần tử, mỗi phần tử là phần tử nhỏ thứ d của một cột. Chọn

$$m = b \circ T.$$

Nếu $n/c > 1$, dùng đệ quy PICK để chọn phần tử này.

2. Tính $m\rho S$: so sánh m với mọi phần tử $x \in S$ mà ta chưa biết $m < x$ hay $m > x$.
3. Nếu $m\rho S = i$, dừng, vì $m = i \circ S$. Nếu $m\rho S > i$, loại

$$D = \{x \mid x \geq m\}$$

và đặt $n \leftarrow n - |D|$. Nếu $m\rho S < i$, loại

$$D = \{x \mid x \leq m\},$$

rồi đặt $n \leftarrow n - |D|$ và $i \leftarrow i - |D|$. Quay lại bước 1.

Sau bước 1, thứ tự bộ phận đã xác định trên S có thể hình dung như sau. Các cột có phần tử lớn nhất ở trên cùng. Tập T gồm các phần tử nhỏ thứ d của các cột. Vì lời gọi đệ quy ở bước 1(b) đã xác định phần tử nào của T nhỏ hơn m , phần tử nào lớn hơn m , các cột được chia thành hai nhóm: $b - 1$ cột có phần tử nhỏ thứ d nhỏ hơn m , và $n/c - b$ cột có phần tử nhỏ thứ d lớn hơn m . Mọi phần tử trong vùng G chắc chắn lớn hơn m , còn mọi phần tử trong vùng L chắc chắn nhỏ hơn m . Vì vậy, ở bước 2 chỉ cần so sánh m với các phần tử trong hai vùng còn chưa quyết định, gọi là A và B .

Không có phần tử nào bị loại sai ở bước 3. Nếu $m\rho S > i$ thì m quá lớn, nên m và mọi phần tử lớn hơn nó đều có thể bị loại; trường hợp $m\rho S < i$ đối xứng. Chú ý rằng ít nhất toàn bộ G hoặc toàn bộ L sẽ bị loại.

Định lý 1.

$$f(i, n) = O(n).$$

Chứng minh. Ta chỉ ra rằng một lựa chọn hợp lý cho $b(i, n)$, $c(i, n)$, và $d(i, n)$ cho một thuật toán chọn thời gian tuyến tính. Gọi $h(c)$ là chi phí sắp xếp c số bằng thuật toán Ford–Johnson. Đã biết rằng

$$h(c) = \sum_{1 \leq j \leq c} \left\lceil \log_2 \frac{3j}{4} \right\rceil. \quad (4)$$

Chi phí của bước 1(a) là $n \cdot h(c)/c$, từ đó thấy ngay rằng $c(i, n)$ phải bị chặn trên bởi một hằng số nếu muốn PICK chạy trong thời gian tuyến tính.

Gọi $P(n)$ là chi phí lớn nhất của PICK trên mọi i . Chi phí của bước 1(b) bị chặn bởi $P(n/c)$. Từ cấu trúc các vùng L và G ở trên, ta có

$$P(n) \leq n \frac{h(c)}{c} + P\left(\frac{n}{c}\right) + n + P(n - \min(|L|, |G|)). \quad (5)$$

Để tối ưu hóa chặn này, chọn $c = 21$, $d = 11$, và $b = n/(2c) = n/42$. Khi đó m là trung vị của T , còn T là tập các trung vị cột. Vì $h(21) = 66$, suy ra

$$P(n) \leq \frac{66n}{21} + P\left(\frac{n}{21}\right) + n + P\left(\frac{31n}{42}\right). \quad (6)$$

Bằng quy nạp toán học, điều này kéo theo

$$P(n) \leq \frac{58n}{3} < 19.4n. \quad (7)$$

Trường hợp cơ sở dựa trên việc $h(n) < 19n$ với $n < 10^5$, nên các trường hợp nhỏ có thể xử lý bằng cách sắp xếp. PICK chạy thời gian tuyến tính vì mỗi lượt loại được một phần đáng kể của S , với chi phí tỉ lệ với số phần tử còn lại trong lượt đó. Chú ý rằng cần $c \geq 5$ để PICK chạy trong thời gian tuyến tính.

3 Cải thiện PICK

Kết quả chính $f(i, n) = O(n)$ đã được chứng minh. Phần này phân tích chi tiết các phiên bản cải tiến của PICK. Bài báo mô tả hai sửa đổi: PICK1, cho cận tổng quát tốt nhất đối với $F(\alpha)$, và PICK2, hiệu quả hơn PICK1 khi $i < \beta n$ hoặc $i > (1 - \beta)n$, với $\beta = 0.203688 \dots$. Nhờ tính đối xứng, trong suốt phần này giả sử $i \leq \lfloor n/2 \rfloor$.

Định lý 2. Với $0 \leq \alpha \leq 1$,

$$F(\alpha) \leq 5.4303.$$

Chứng minh. Ta phân tích PICK1, khác PICK ở các điểm sau.

1. Các phần tử của S chỉ được sắp vào cột một lần. Sau đó, những cột bị phá vỡ bởi thao tác loại bỏ được khôi phục về độ dài đầy đủ bằng một bước trộn mới ở cuối mỗi lượt.
2. Bước phân hoạch được sửa để số phép so sánh là một hàm tuyến tính của số phần tử cuối cùng bị loại.
3. Thao tác loại bỏ không phá vỡ quá nửa số cột trong mỗi lượt, nhờ đó các sửa đổi khác hoạt động tốt.
4. Bước sắp xếp ngầm trong lời gọi đệ quy để chọn m được thay một phần bằng bước trộn kể từ lượt thứ hai trở đi, vì điểm trước bảo đảm rằng một nửa tập T ở lượt j cũng đã xuất hiện trong lời gọi đệ quy ở lượt $j - 1$.

Một k -cột là một cột đã sắp có độ dài k . Giá trị tối ưu của hàm c là 15, và sẽ được dùng trực tiếp.

Thủ tục PICK1. Chọn $i \in S$, với $|S| = n$ và $1 \leq i \leq \lfloor n/2 \rfloor$.

1. Nếu $n \leq 45$, sắp S , in $i \in S$, rồi dừng.
2. Sắp S thành $\lfloor n/15 \rfloor$ cột 15-phần tử, và có thể thêm một cột gồm $n \bmod 15$ phần tử.
3. Dùng thủ tục PICK1A để chọn $i \in S$.

Thủ tục PICK1A. Giống PICK1, ngoại trừ việc S đã được sắp thành các cột 15-phần tử.

1. Nếu $n \leq 45$, sắp S , in $i \in S$, rồi dừng.
2. Sắp tập T gồm các trung vị cột thành các cột 15-phần tử, và có thể thêm một cột gồm $\lfloor n/15 \rfloor \bmod 15$ phần tử.
3. Dùng thủ tục PICK1B để chọn $i \in S$.

Thủ tục PICK1B. Giống PICK1A, ngoại trừ việc T cũng đã được sắp thành các cột 15-phần tử.

1. Dùng PICK1A để chọn m , trung vị của T .
2. Phân hoạch $A \cup B$ quanh m , và dừng ngay khi đã rõ $m \rho S < i$ hoặc $m \rho S > i$:
 - (a) Chèn m vào mỗi 7-cột của B bằng chèn nhị phân, tốn 3 phép so sánh mỗi cột.
 - (b) Chèn m vào mỗi 7-cột của A bằng tìm kiếm tuyến tính bắt đầu gần trung vị của từng cột 15-phần tử.

3. Nếu $m\rho S = i$, in $m = i \circ S$, rồi dừng. Nếu $m\rho S > i$, loại

$$G \cup \{x \mid x \in B, x > m\}.$$

Nếu $m\rho S < i$, loại

$$L \cup \{x \mid x \in A, x < m\},$$

rồi giảm i đi số phần tử bị loại.

4. Khôi phục S thành một tập các cột 15-phần tử bằng các phép trộn sau. Gọi $|X|$ là số phần tử của một tập X . Gọi U là tập các cột có độ dài nhỏ hơn 15; thực tế, mỗi cột của U có độ dài không quá 7. Gọi $Y \subset U$ là tập các cột ngắn nhất của U , sao cho

$$|Y| = |U| \bmod 15,$$

và gọi V là tập mọi 7-cột trong $U - Y$. Chia $U - (V \cup Y)$ thành hai tập con X và W , sao cho W chứa w cột, các cột của W không ngắn hơn các cột của X , và

$$|W| + |X| = 7w.$$

Sau đó:

- (a) Kéo dài mọi cột trong W lên độ dài 7 bằng cách dùng chèn nhị phân để đặt từng phần tử của X vào một cột của W .
 - (b) Khi đó mọi cột trong $U - Y$ đều là 7-cột. Trộn chúng từng cặp để tạo thành các 14-cột.
 - (c) Dùng chèn nhị phân để đặt từng phần tử của Y vào một 14-cột. Khi đó S đã được khôi phục thành một tập các cột 15-phần tử.
5. Khôi phục tập T các trung vị cột thành các cột 15-phần tử. Gọi $Z \subset T$ là các trung vị cột đã là trung vị cột ở bước 1. Các phần tử của Z đã được sắp thành các cột có kích thước ít nhất 8, vì bước 3 của lời gọi đệ quy ở bước 1 đã loại Z theo các chuỗi có kích thước như vậy.

- (a) Trộn các cột của Z để tạo thành các cột 15-phần tử và một số phần dư, xử lý riêng từng kích thước cột:

$$\begin{aligned} 2(8) &: 8 + 7, & 1 \text{ phần dư,} \\ 5(9) &: 9 + 6, & 9 + 6, & 9 + 3 + 3, & 0 \text{ phần dư,} \\ 3(10) &: 10 + 5, & 10 + 5, & 0 \text{ phần dư,} \\ 4(11) + 1(1) &: 11 + 4, & 11 + 4, & 11 + 3 + 1, & 0 \text{ phần dư.} \end{aligned}$$

Với 11-cột, trước tiên tách riêng $1/45$ số 11-cột và phá chúng thành các 1-cột. Với các cột độ dài 12 trở lên, tách riêng một số phần tử để chèn nhị phân vào các cột còn lại. Cuối cùng, sắp các phần dư thành các cột 15-phần tử.

- (b) Sắp $T - Z$ thành các cột 15-phần tử. Lúc này T đã được khôi phục thành một tập các cột 15-phần tử.
6. Giảm n đi số phần tử bị loại ở bước 5. Nếu $n \leq 45$, sắp S , in $i \circ S$, rồi dừng; nếu không, quay lại bước 1.

Để phân tích PICK1, dùng các ký hiệu sau:

$$\begin{aligned} P_1(n), P_{1a}(n), P_{1b}(n) &= \text{chi phí lớn nhất của PICK1, PICK1A, PICK1B,} \\ v &= \text{số phép so sánh trong bước PICK1B(2ii),} \\ d &= \text{số phần tử từ } A \cup B \text{ bị loại ở bước PICK1B(3),} \\ g_a, g_b &= \text{số phần tử từ } A, B \text{ được thấy là lớn hơn } m, \\ l_a, l_b &= \text{số phần tử từ } A, B \text{ được thấy là nhỏ hơn } m, \\ w, x, y &= \text{số cột trong các tập } W, X, Y \text{ ở bước PICK1B(4).} \end{aligned}$$

Vì $h(15) = 42$, ta có ngay

$$P_1(n) \leq \frac{42n}{15} + P_{1a}(n) = 2.8n + P_{1a}(n), \quad (8)$$

và

$$P_{1a}(n) \leq \frac{42n}{225} + P_{1b}(n) = 0.186\bar{6}n + P_{1b}(n). \quad (9)$$

Bổ đề 1.

$$v \leq d + \frac{n}{30}. \quad (10)$$

Chứng minh. Có hai trường hợp, tùy theo bước PICK_{1B}(3) loại L hay G .

Nếu L bị loại, thì rõ ràng có nhiều nhất một phép so sánh cho mỗi cột trong A , cộng thêm một phép cho mỗi phần tử của A bị loại.

Nếu G bị loại, thì từ điều kiện dừng của phân hoạch và do $i \leq \lfloor n/2 \rfloor$, suy ra số phần tử của B được chứng minh lớn hơn m không nhỏ hơn số phần tử tương ứng cần để trả cho các phép so sánh tuyến tính trong A . Nói cách khác, $g_b \geq l_a$, còn $l_a \geq v - n/30$ như trường hợp trước. Vì trong trường hợp này $d = g_b$, ta được bổ đề.

Bổ đề 2.

$$|X| \leq \frac{6d}{7}. \quad (11)$$

Chứng minh. Từ cách chọn X, W, Y ,

$$d \geq 7(w + x + y) - |W| - |X| - |Y|,$$

và $7w - |W| = |X|$. Do đó

$$d \geq 7x + 7y - |Y| \geq 7x.$$

Vì $6x \geq |X|$, suy ra $d \geq 7|X|/6$.

Bước PICK_{1B}(5i) tốn trong trường hợp xấu nhất $21/20$ phép so sánh trên mỗi phần tử để trộn và sắp Z thành các cột 15-phần tử; trường hợp xấu này xảy ra khi Z chỉ chứa các 8-cột. Vì $|Z| = n/30$, bước này tốn nhiều nhất $7n/200$ phép so sánh.

Từ đó có truy hồi

$$\begin{aligned} P_{1b}(n) &\leq P_{1a}(\lfloor n/15 \rfloor) + 3(n/30) + (d + n/30) + 3(6d/7) \\ &\quad + \frac{13(7n/30 - d)}{15} + \frac{4(7n/30 - d)}{15} + \frac{7n}{200} \\ &\quad + \frac{42(7n/30 - d)}{225} + P_{1b}(11n/15 - d). \end{aligned}$$

Các hạng tử lần lượt ứng với các bước 1, 2i, 2ii, 4i, 4ii, 4iii, 5i, 5ii, và các lượt tiếp theo. Rút gọn cho

$$P_{1b}(n) \leq (n + 5d) + P_{1b}\left(\frac{13197n}{27000} - d\right). \quad (12)$$

Vế phải của (12) lớn nhất tại $d = 0$, nên

$$P_{1b}(n) \leq \frac{13197n}{27000 - 13197} < 2.443n, \quad (13)$$

$$P_{1a}(n) < 2.6305n, \quad (14)$$

và

$$P_1(n) < 5.4305n. \quad (15)$$

Vì

$$\max_{1 \leq c \leq 45} \frac{h(c)}{c} = \frac{h(45)}{45} < 5.43, \quad (16)$$

cơ sở quy nạp cho (12) là hợp lệ; điều này cũng xử lý các bước PICK1(1), PICK1A(1), và PICK1B(6). Định lý được chứng minh.

Định lý 3. Với $0 < \alpha \leq \beta$,

$$F(\alpha) \leq 1 + 4.4303 \frac{\alpha}{\beta} + 10.861\alpha \lceil \log_2(\beta/\alpha) \rceil, \quad (17)$$

trong đó $\beta = 0.203688 \dots$

Chứng minh. Ta phân tích PICK2, về bản chất giống PICK với

$$b(i, n) = i, \quad c(i, n) = 2, \quad d(i, n) = 1,$$

và bỏ bước phân hoạch.

Thủ tục PICK2. Chọn $i \in S$, với $|S| = n$ và $i < \beta n$.

1. So sánh các phần tử của S từng cặp, tạo $\lfloor n/2 \rfloor$ cặp và có thể một phần tử lẻ.
2. Gọi T là tập các phần tử nhỏ hơn trong mỗi cặp. Nếu $i < \beta \lfloor n/2 \rfloor$, dùng PICK2; nếu không, dùng PICK1 để chọn m , phần tử nhỏ thứ i của T .
3. Loại mọi phần tử đã biết là lớn hơn m . Cụ thể, trong các cặp mà phần tử nhỏ hơn đã lớn hơn m , cả hai phần tử đều lớn hơn m và bị loại.
4. Dùng PICK1 để chọn $i \in S$ từ phần còn lại của S .

Khi $i = 1$, thủ tục này trở thành một giải loại trực tiếp đơn giản. Gọi $P_2(i, n)$ là số phép so sánh lớn nhất mà PICK2 dùng để chọn i . Ta có

$$P_2(i, n) \leq \lfloor n/2 \rfloor + \min\{P_1(\lfloor n/2 \rfloor), P_2(i, \lfloor n/2 \rfloor)\} + P_1(2i). \quad (18)$$

Với các giá trị cụ thể của i và n , thủ tục PICK2 được gọi liên tiếp

$$t = \lceil \log_2(\beta n/i) \rceil$$

lần trong các lời gọi đệ quy ở bước 2 trước khi PICK1 được gọi. Vì vậy

$$P_2(i, n) \leq \sum_{0 \leq j < t} \left\lfloor \frac{n}{2^{j+1}} \right\rfloor + P_1\left(\frac{n}{2^t}\right) + tP_1(2i). \quad (19)$$

Bất đẳng thức này trực tiếp suy ra định lý. Giá trị đúng của $\beta = 0.203688 \dots$ là giá trị lớn nhất sao cho

$$P_2(\lfloor \beta n \rfloor, n) < P_1(n).$$

Các kết quả của phần này được tóm tắt trong hình của bản gốc: đường cận trên đối với $F(\alpha)$ dùng PICK2 khi α gần 0 hoặc gần 1, và dùng PICK1 ở vùng giữa. Bài báo nhận xét rằng không vô lý khi phỏng đoán đường cong thật của $F(\alpha)$ là đơn đỉnh và đạt cực đại tại $\alpha = 1/2$. Độ phức tạp tương đối của PICK1 cũng khiến các tác giả phỏng đoán rằng cận trên còn có thể được cải thiện đáng kể.

4 Một cận dưới

Phần này suy ra một cận dưới cho $F(\alpha)$ bằng phương pháp “đối thủ”. Knuth gọi kỹ thuật này là xây dựng một “oracle”. Quá trình chọn có thể được mô hình hóa như một trò chơi giữa thuật toán chọn, gọi là người chơi A , và đối thủ của nó, gọi là người chơi B . Người chơi A muốn tìm $i \in S$ với ít phép so sánh nhất; người chơi B muốn buộc A dùng nhiều phép so sánh nhất. Hai người đi luân phiên. Mỗi nước đi của A là một câu hỏi so sánh, chẳng hạn “ $x < y$ hay không?” với $x, y \in S$. Ở lượt của mình, B phải trả lời “có” hoặc “không”. Các câu trả lời của B có thể tùy ý, miễn là không mâu thuẫn với các câu trả lời trước đó. Khi A đã rút ra đủ thông tin để xác định $i \in S$, trò chơi kết thúc.

Lợi thế của cách nhìn này là một cận dưới không tầm thường cho độ dài trò chơi, độc lập với chiến lược của A , có thể thu được chỉ bằng cách đưa ra một chiến lược đủ khéo cho B . Độ dài trò chơi ở đây chính là chi phí minimax thông thường:

$$f(i, n) := \min_A \max_B c(A, B), \quad (20)$$

trong đó $c(A, B)$ là độ dài trò chơi giữa hai chiến lược cụ thể A và B .

Trong trò chơi này, B đóng vai trò của dữ liệu. Một chiến lược của B thực chất là một quy tắc tính ra một bộ dữ liệu đặc biệt xấu cho chiến lược của A , vì luôn có thể dựng một tập số thật phù hợp với các câu trả lời của B . Do đó một chiến lược tốt cho B là một thủ tục làm khó bất kỳ chiến lược nào của A .

Ta mô tả chiến lược cụ thể của B để thu được cận dưới. Khi trò chơi diễn ra, sẽ có nhiều phần tử x mà A đã biết đủ để kết luận $x \neq i_0$, tức $x < i_0$ hoặc $x > i_0$. Ban đầu, B xem mọi phần tử $x \in S$ là thành viên của tập U , nghĩa là B , và do đó cả A , còn chưa chắc liệu $x < i_0$, $x = i_0$, hay $x > i_0$. Sau một thời gian, A có thể buộc B quyết định trạng thái của một phần tử cụ thể $x \in U$. Nếu B quyết định $x > i_0$, B bỏ x khỏi U và đưa vào tập G . Tương tự, nếu B quyết định $x < i_0$, B bỏ x khỏi U và đưa vào tập L . Ban đầu G và L đều rỗng. Phần tử cuối cùng là i_0 sẽ là một phần tử nào đó còn trong U , nên chừng nào $|U| > 1$ thì trò chơi còn phải tiếp tục. Chiến lược của B cố giữ U càng lớn càng lâu càng tốt.

Theo chiến lược của B , trò chơi gồm hai pha. Chừng nào

$$|L| < i - 1 \quad \text{và} \quad |G| < n - i,$$

B còn toàn quyền đưa một phần tử $x \in U$ vào L hoặc G . Sau khi một trong hai tập L hoặc G đầy, B bị ràng buộc hơn nhiều, vì không được phép làm $|L| \geq i$ hoặc $|G| \geq n - i + 1$. Khi đó trò chơi suy biến thành việc A chỉ còn phải tìm phần tử nhỏ nhất hoặc lớn nhất của U .

Trong pha thứ nhất, B không bao giờ bỏ quá một phần tử khỏi U trong một lượt. Điều này không gây rắc rối nếu phần tử đó là cực đại hoặc cực tiểu của U , và B đưa nó vào G hoặc L tương ứng. Mỗi phần tử đưa vào G được giả sử là nhỏ hơn mọi phần tử đã đưa vào G trước đó, đồng thời lớn hơn mọi phần tử còn trong U và L . Đối xứng, mỗi phần tử đưa vào L được giả sử là lớn hơn mọi phần tử đã đưa vào L trước đó, đồng thời nhỏ hơn mọi phần tử còn trong U và G . Quy tắc này xác định hoàn toàn câu trả lời của B , ngoại trừ khi A muốn so sánh hai phần tử $x, y \in U$. Ngoài ra, B chỉ bỏ một phần tử khỏi U khi A đưa ra một yêu cầu như vậy.

B luôn hạn chế U sao cho mỗi phần tử $x \in U$ là cực đại hoặc cực tiểu của U , hoặc cả hai, đối với thứ tự bộ phận đã được các câu trả lời trước đó của B cố định trên S . Thậm chí, B duy trì điều kiện mạnh hơn: với mỗi $x \in U$, có nhiều nhất một $y \in U$ sao cho đã biết $x < y$ hoặc $y < x$. Vì vậy U chỉ có ba trạng thái phần tử. Đặt $\varepsilon(x)$ bằng -1 , 0 , hoặc 1 , tùy theo x là phần tử nhỏ hơn trong một cặp, là phần tử cô lập, hay là phần tử lớn hơn trong một cặp.

Chiến lược của B khi so sánh hai phần tử $x, y \in U$ là như sau; không mất tính tổng quát, giả sử $\varepsilon(x) \leq \varepsilon(y)$.

1. Trả lời rằng x nhỏ hơn y .

2. Nếu $\varepsilon(x) = \varepsilon(y) = 0$, không làm gì thêm. Nếu $\varepsilon(x) = -1$, bỏ x khỏi U và đưa vào L . Ngược lại, bỏ y khỏi U và đưa vào G .

Về bản chất, chiến lược của B tạo một cặp mới trong U khi $\varepsilon(x) = \varepsilon(y) = 0$; trong các trường hợp khác, một phần tử bị bỏ khỏi U , và số cặp trong U giảm một. Gọi

$$c = \text{số phép so sánh đã thực hiện}, \quad p = \text{số cặp hiện có trong } U.$$

Dễ kiểm tra rằng, chừng nào trò chơi còn ở pha thứ nhất, chiến lược của B duy trì điều kiện

$$c - p + 2|U| = 2n. \quad (21)$$

Điều này đúng lúc bắt đầu, khi $c = p = 0$ và $|U| = n$.

Khi pha thứ nhất kết thúc, L hoặc G đã đầy, nên

$$|U| \geq n - \min(i - 1, n - i). \quad (22)$$

Hơn nữa, trong pha thứ hai, A cần ít nhất $|U| - 1 - p$ phép so sánh nữa để kết thúc trò chơi: tối thiểu A phải làm công việc tìm phần tử nhỏ nhất hoặc lớn nhất của U , vốn cần $|U| - 1$ phép so sánh, trong đó p phép đã được thực hiện. Vì vậy tổng số phép so sánh ít nhất là

$$f(i, n) \geq c + |U| - 1 - p \geq n + \min(i - 1, n - i) - 1, \quad 1 \leq i \leq n. \quad (23)$$

Cho $n \rightarrow \infty$ với $i = \lfloor \alpha(n - 1) \rfloor + 1$, ta được

$$F(\alpha) \geq 1 + \min(\alpha, 1 - \alpha). \quad (24)$$

Cận này cũng được vẽ trong hình tổng kết của bản gốc.

5 Tóm tắt

Kết quả quan trọng nhất của bài báo là: trong trường hợp xấu nhất, bài toán chọn có thể thực hiện trong thời gian tuyến tính. Không cần quá $5.4305n$ phép so sánh để chọn phần tử nhỏ thứ i trong n số, với mọi $1 \leq i \leq n$. Cận này có thể cải thiện khi i gần hai đầu khoảng giá trị của nó.

Bài báo cũng đưa ra một cận dưới tổng quát, đặc biệt cho thấy cần ít nhất

$$\frac{3n}{2} - 2$$

phép so sánh để tính trung vị. Các tác giả tin rằng các hằng số tỉ lệ trong cả cận trên lẫn cận dưới đều có thể được cải thiện đáng kể.

Tài liệu tham khảo

1. Lewis Carroll, "Lawn Tennis Tournaments," *St. James's Gazette* (August 1, 1883), pp. 5–6. Reprinted in *The Complete Works of Lewis Carroll*, New York Modern Library, 1947.
2. L. R. Ford and S. M. Johnson, A tournament problem, *The American Mathematical Monthly* 66 (May 1959), 387–389.
3. Abdollah Hadian, Optimality properties of various procedures for ranking n different numbers using only binary comparisons, Technical Report 117, Dept. of Statistics, Univ. of Minnesota, May 1969, Ph.D. thesis.

4. Abdollah Hadian and Milton Sobel, Selecting the t -th largest using binary errorless comparisons, Technical Report 121, Dept. of Statistics, Univ. of Minnesota, May 1969.
5. C. A. R. Hoare, Find (Algorithm 65), *Communications of the ACM*, July 1961, pp. 321–322.
6. S. S. Kislitsin, On the selection of the k -th element of an ordered set by pairwise comparisons, *Sibirsk. Math. Z.* 5 (1964), 557–564 (MR 29, No. 2198), Russian.
7. Donald E. Knuth, *The Art of Computer Programming*, Vol. III: Sorting and Searching, Addison-Wesley, 1973.
8. Józef Schreier, O systemach eliminacji w turniejach (On elimination systems in tournaments), *Mathesis Polska* 7 (1932), 154–160, Polish.